VULNERABILITY ASSESSMENT REPORT

For a Live Website — Read-Only Scope

Performed and Tested by : SHABBIR AHMADI

Target Website:

https://demo.testfire.net | http://testphp.vulnweb.com

Tools Used:

OWASP ZAP (Passive Scan) | SecurityHeaders.com | Nmap

Prepared For:

Future Interns — Cyber Security Internship | Task 1

Date: 29 April 2026

Scope: Read-Only | Passive Analysis | Ethical Testing

1. Executive Summary

This Vulnerability Assessment Report presents the findings of a passive, read-only security analysis conducted on two publicly accessible demo websites: https://demo.testfire.net (AltoroMutual banking demo) and http://testphp.vulnweb.com. The assessment was conducted on 29 April 2026 as part of the Future Interns Cyber Security Internship — Task 1.

The analysis identified several critical and high-risk vulnerabilities including SQL Injection, Cross-Site Scripting (XSS), missing HTTP security headers, and exposed server information. These vulnerabilities, if present in a real production environment, could lead to data breaches, user account compromise, and significant business and reputational damage.

This report classifies all findings by risk level and provides clear, actionable remediation steps written in business-friendly language.

2. Scope & Methodology

Target 1: https://demo.testfire.net (OWASP ZAP scan + Security Headers check)
Target 2: http://testphp.vulnweb.com (Nmap port scan — demo.testfire.net blocks Nmap)
Scan Date: 29 April 2026
Scan Type: Passive / Read-Only — No exploitation performed

Tools Used

OWASP ZAP: Automated passive vulnerability scanner
SecurityHeaders.com: HTTP security header analysis
Nmap 7.99: Port and service discovery

Ethical Boundary

All testing was strictly limited to passive observation and scanning of publicly accessible pages. No login bypass, brute force, denial-of-service, or active exploitation was performed. This assessment follows ethical security auditing guidelines.

Note on Nmap Target

Nmap was run against testphp.vulnweb.com instead of demo.testfire.net because demo.testfire.net has firewall rules that block port-scanning tools. This is itself a positive security measure. testphp.vulnweb.com is a deliberately vulnerable practice site that permits scanning.

3. Risk Classification Summary

The following table summarises all vulnerabilities identified during the assessment:

Vulnerability	Target	Risk Level	Tool
SQL Injection	demo.testfire.net	**High**	OWASP ZAP
Cross-Site Scripting — DOM Based (50 instances)	demo.testfire.net	**High**	OWASP ZAP
Cross-Site Scripting — Reflected (2 instances)	demo.testfire.net	**High**	OWASP ZAP
Absence of Anti-CSRF Tokens	demo.testfire.net	**Medium**	OWASP ZAP
Content Security Policy (CSP) Header Not Set	demo.testfire.net	**Medium**	OWASP ZAP / SecurityHeaders
Missing Strict-Transport-Security Header	demo.testfire.net	**Medium**	OWASP ZAP / SecurityHeaders
Missing X-Frame-Options Header	demo.testfire.net	**Medium**	SecurityHeaders
Missing X-Content-Type-Options Header	demo.testfire.net	**Medium**	OWASP ZAP / SecurityHeaders
Missing Referrer-Policy Header	demo.testfire.net	**Low**	SecurityHeaders
Missing Permissions-Policy Header	demo.testfire.net	**Low**	SecurityHeaders
Server Version Leaked in HTTP Header	demo.testfire.net	**Medium**	OWASP ZAP / SecurityHeaders
Cookie Without SameSite Attribute (3)	demo.testfire.net	**Low**	OWASP ZAP
Sub Resource Integrity Attribute Missing	demo.testfire.net	**Low**	OWASP ZAP
Cross-Domain JavaScript Source File Inclusion	demo.testfire.net	**Low**	OWASP ZAP
HTTPS Content Available via HTTP	demo.testfire.net	**Medium**	OWASP ZAP
Information Disclosure — Suspicious Comments (6)	demo.testfire.net	**Informational**	OWASP ZAP
All 1000 Ports Filtered / No Open Ports Found	testphp.vulnweb.com	**Informational**	Nmap

4. Detailed Findings

4.1 OWASP ZAP — Automated Passive Scan (demo.testfire.net)

OWASP ZAP was run in automated scan mode against https://demo.testfire.net. The tool identified 20 distinct alert types. The screenshots below show the full alert list and the SQL Injection detail panel.

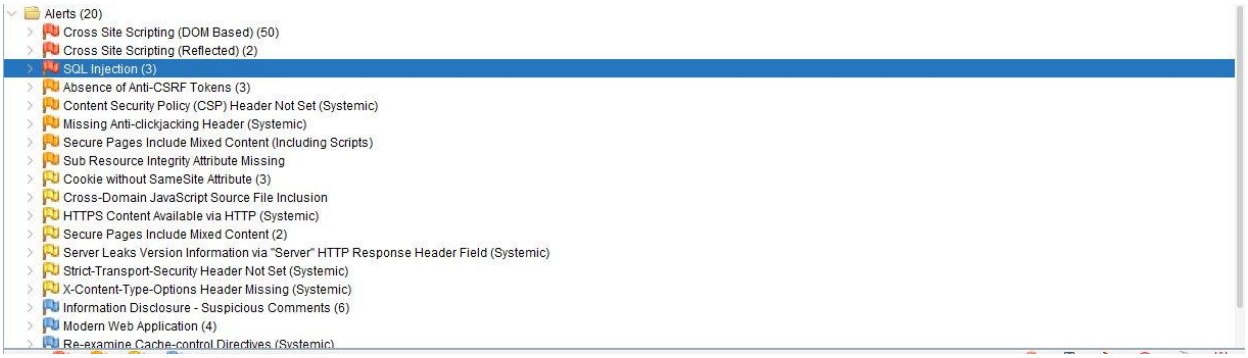

Figure 1: OWASP ZAP — Full Alert List

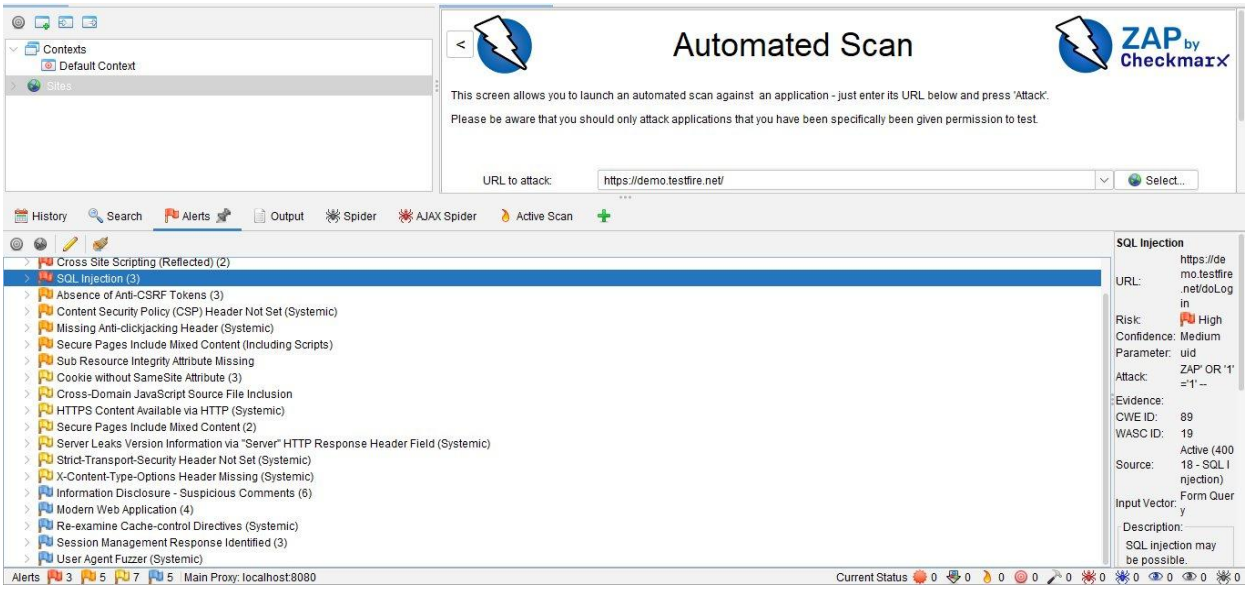

Figure 2: OWASP ZAP — SQL Injection Detail (Risk: High, CWE-89)

F-01 — SQL Injection Risk: **High**

Target: demo.testfire.net/doLogin

What is it: SQL Injection is a critical vulnerability where an attacker can insert malicious database commands into input fields (like username or password boxes). ZAP confirmed this by successfully injecting a test payload: ZAP OR '1'='1'.

Why it matters: An attacker can use this to log in without a password, steal all user data from the database, delete records, or take over the server.

How to fix: Use parameterised queries / prepared statements in all database calls. Never directly insert user input into SQL queries. Implement a Web Application Firewall (WAF).

F-02 — Cross-Site Scripting (XSS) — DOM Based & Reflected Risk: **High**

Target: demo.testfire.net

What is it: XSS allows attackers to inject malicious JavaScript code into web pages that other users see. DOM-based XSS (50 instances) runs in the browser, while Reflected XSS (2 instances) is triggered via crafted URLs.

Why it matters: Attackers can steal user session cookies, redirect users to fake phishing pages, or silently take over a user's browser session.

How to fix: Sanitise and encode all user-supplied input before displaying it on a page. Implement Content Security Policy (CSP) headers. Use modern framework protections that escape output by default.

F-03 — Absence of Anti-CSRF Tokens Risk: **Medium**

Target: demo.testfire.net

What is it: CSRF (Cross-Site Request Forgery) allows an attacker to trick a logged-in user into unknowingly submitting a form or taking an action on the website.

Why it matters: For a banking demo site, this could mean an attacker tricks a user into submitting a fund transfer without their knowledge.

How to fix: Add a unique, secret CSRF token to every form submission and validate it on the server side.

F-04 — Server Version Leaked in HTTP Header Risk: **Medium**

Target: demo.testfire.net

What is it: The server reveals it is running Apache-Coyote/1.1 in its HTTP response headers. This was confirmed in the Security Headers raw header scan.

Why it matters: Knowing the exact server software and version allows attackers to look up known vulnerabilities for that specific version and target them.

How to fix: Configure the web server to suppress or mask the Server header value. Remove version information from all response headers.

4.2 Security Headers Check (SecurityHeaders.com)

The website https://demo.testfire.net was scanned using securityheaders.com on 29 April 2026. The site received a grade of F, indicating critical security header misconfigurations.

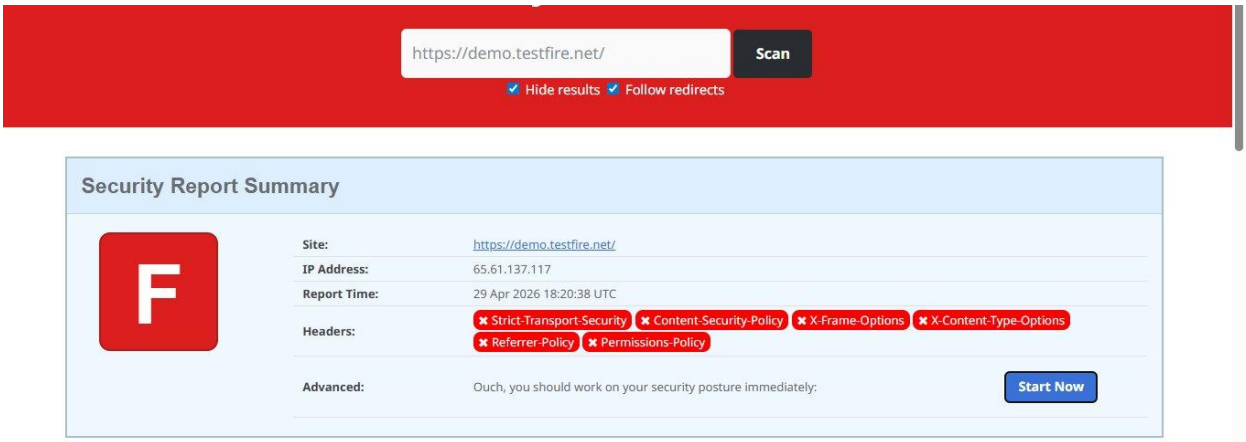

Figure 3: Security Headers — Grade F with 5 critical missing headers

Missing Headers

Strict-Transport-Security	HTTP Strict Transport Security is an excellent feature to support on your site and strengthens your implementation of TLS by getting the User Agent to enforce the use of HTTPS. Recommended value "Strict-Transport-Security: max-age=31536000; includeSubDomains".
Content-Security-Policy	Content Security Policy is an effective measure to protect your site from XSS attacks. By whitelisting sources of approved content, you can prevent the browser from loading malicious assets.
X-Frame-Options	X-Frame-Options tells the browser whether you want to allow your site to be framed or not. By preventing a browser from framing your site you can defend against attacks like clickjacking. Recommended value "X-Frame-Options: SAMEORIGIN".
X-Content-Type-Options	X-Content-Type-Options stops a browser from trying to MIME-sniff the content type and forces it to stick with the declared content-type. The only valid value for this header is "X-Content-Type-Options: nosniff".
Referrer-Policy	Referrer Policy is a new header that allows a site to control how much information the browser includes with navigations away from a document and should be set by all sites.
Permissions-Policy	Permissions Policy is a new header that allows a site to control which features and APIs can be used in the browser.

Figure 4: Missing Security Headers Detail

Raw Headers

HTTP/1.1	200 OK
Server	Apache-Coyote/1.1
Set-Cookie	JSESSIONID=76040D279C35D4D14DEA4D817B03D1C3; Path=/; **Secure**; **HttpOnly**
Content-Type	text/html;charset=ISO-8859-1
Transfer-Encoding	chunked
Date	Wed, 29 Apr 2026 18:20:37 GMT

Upcoming Headers

Cross-Origin-Embedder-Policy	Cross-Origin Embedder Policy allows a site to prevent assets being loaded that do not grant permission to load them via CORS or CORP.
Cross-Origin-Opener-Policy	Cross-Origin Opener Policy allows a site to opt-in to Cross-Origin Isolation in the browser.
Cross-Origin-Resource-Policy	Cross-Origin Resource Policy allows a resource owner to specify who can load the resource.

Figure 5: Raw HTTP Response Headers — Server version exposed (Apache-Coyote/1.1)

Missing Headers Summary

Strict-Transport-Security: Forces browsers to always use HTTPS. Without it, users can be redirected to unencrypted HTTP.

Content-Security-Policy: Prevents XSS attacks by controlling which scripts can run on the page.

X-Frame-Options: Prevents the site from being embedded in iframes, protecting against clickjacking attacks.

X-Content-Type-Options: Stops browsers from guessing file types, preventing MIME-type confusion attacks.

Referrer-Policy: Controls how much information is shared when users click links off the site.

Permissions-Policy: Controls access to browser features like camera, microphone, and location.

4.3 Nmap Port Scan (testphp.vulnweb.com)

Nmap version 7.99 was used to perform a basic port scan on testphp.vulnweb.com. Note: demo.testfire.net was not scanned with Nmap because its firewall actively blocks port-scanning tools, which is a positive security behaviour.

```
C:\Windows\System32>nmap testphp.vulnweb.com
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-29 23:53 +0530
Nmap scan report for testphp.vulnweb.com (44.228.249.3)
Host is up (0.047s latency).
Other addresses for testphp.vulnweb.com (not scanned): 64:ff9b::2ce4:f903
rDNS record for 44.228.249.3: ec2-44-228-249-3.us-west-2.compute.amazonaws.com
All 1000 scanned ports on testphp.vulnweb.com (44.228.249.3) are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)

Nmap done: 1 IP address (1 host up) scanned in 14.15 seconds
```

Figure 6: Nmap Scan — All 1000 ports filtered on testphp.vulnweb.com

Finding: All 1000 scanned TCP ports are in filtered/no-response state

IP Address: 44.228.249.3 (AWS EC2 — us-west-2)

Meaning: The server has a firewall that is blocking all standard port probes. This is actually a good security practice — it reduces the attack surface by hiding open services from external scanners.

Recommendation: Continue maintaining firewall rules. Ensure only necessary ports are open and that internal monitoring is in place.

5. Remediation Recommendations

The table below provides prioritised remediation steps. High-risk items should be addressed immediately.

Issue	Risk	Priority	Recommended Fix
SQL Injection	**High**	Immediate	Use prepared statements / parameterised queries. Deploy a WAF.
Cross-Site Scripting (XSS)	**High**	Immediate	Sanitise all user inputs. Implement CSP headers. Use output encoding.
Add CSP Header	**Medium**	Within 1 week	Set Content-Security-Policy header to whitelist trusted script sources.
Add HSTS Header	**Medium**	Within 1 week	Set Strict-Transport-Security: max-age=31536000; includeSubDomains.
Add X-Frame-Options	**Medium**	Within 1 week	Set X-Frame-Options: SAMEORIGIN to prevent clickjacking.
Add X-Content-Type-Options	**Medium**	Within 1 week	Set X-Content-Type-Options: nosniff.
Implement CSRF Tokens	**Medium**	Within 2 weeks	Add unique CSRF tokens to all forms and validate server-side.
Remove Server Header	**Medium**	Within 2 weeks	Configure Apache/server to suppress version info in HTTP headers.
Add Referrer-Policy	**Low**	Within 1 month	Set Referrer-Policy: strict-origin-when-cross-origin.
Add Permissions-Policy	**Low**	Within 1 month	Define which browser features the site needs access to.

Issue	Risk	Priority	Recommended Fix
Fix SameSite Cookie Attribute	**Low**	Within 1 month	Add SameSite=Strict or SameSite=Lax to all session cookies.

6. Conclusion

The passive security assessment of https://demo.testfire.net revealed a significant number of critical and medium-risk vulnerabilities, including SQL Injection, Cross-Site Scripting, and multiple missing HTTP security headers. The site received an overall security header grade of F, indicating that basic browser-level protections are not in place.

The Nmap scan of testphp.vulnweb.com showed all ports filtered, which is a positive indicator of firewall protection. Similarly, the fact that demo.testfire.net blocked the Nmap scan demonstrates that some protective measures are in place at the network level.

However, the application-level vulnerabilities (SQL Injection and XSS in particular) are critical issues that, in a real-world production website, would expose user data, allow unauthorised access, and potentially lead to complete system compromise. These must be addressed as the highest priority.

This report was prepared for educational and internship purposes as part of Future Interns — Cyber Security Track, Task 1. All testing was conducted ethically within the defined read-only scope on publicly available demo/test websites.